

SQL & Database Fundamentals

Q1. What is SQL in simple words?

A: **SQL (Structured Query Language)** is the standard language used to talk to databases. It allows you to manage data, define database structures, and ask questions of the data (like asking for a list of all customers who bought a specific product).

Q2. Where do we normally use SQL?

A: We normally use SQL with **Relational Database Management Systems (RDBMS)** like MySQL, PostgreSQL, SQL Server, and Oracle. It is used in nearly all business applications, websites, and data analysis tools that rely on structured, organized data tables.

Q3. What are the different types of SQL commands (DML, DDL, DCL, TCL)?

A: There are four main types:

- **DML (Data Manipulation Language):** Used for managing data (SELECT, INSERT, UPDATE, DELETE).
- **DDL (Data Definition Language):** Used for defining structures (CREATE, ALTER, DROP).
- **DCL (Data Control Language):** Used for managing permissions (GRANT, REVOKE).
- **TCL (Transaction Control Language):** Used for managing transactions (COMMIT, ROLLBACK, SAVEPOINT).

Q4. What is a join used for in SQL?

A: A **join** is used to combine rows from two or more tables based on a related column between them (like a common ID). It allows you to retrieve a complete set of related information, such as finding a customer's name and all the orders associated with their customer ID.

Q5. What does the SELECT statement do in MySQL?

A: The **SELECT** statement is the primary DML command used to retrieve data from one or more tables in the database. It allows you to specify exactly which columns you want, which tables to pull from, and what criteria rows must meet.

Q6. What is normalization and why do we use it?

A: **Normalization** is the process of organizing the columns and tables in a relational database to minimize **data redundancy** (duplication). We use it to ensure data dependencies are logical and data integrity is maintained, making the database more flexible, smaller, and easier to update.

Q7. What common data types are available in MySQL?

A: Common data types include **VARCHAR** (for variable-length strings), **INT** (for whole numbers), **DECIMAL** (for precise numeric values like currency), **DATE** (for calendar dates), and **DATETIME** (for date and time combinations).

Q8. How is a primary key different from a unique key?

A: A **Primary Key** uniquely identifies every record in a table and **cannot contain NULL values**. A table can only have one Primary Key. A **Unique Key** also ensures uniqueness for the column but **can allow one NULL value** and a table can have multiple Unique Keys.

Q9. What is a foreign key and what does it enforce?

A: A **Foreign Key** is a column or set of columns in one table that links to the Primary Key of another table. It enforces **referential integrity**, meaning you cannot create a record in the first table if the value in the Foreign Key doesn't exist in the referenced table.

Q10. What is the difference between NULL and zero?

A: **NULL** means the value is unknown or missing; it is not a value itself. **Zero (0)** is a specific numeric value. In SQL, you cannot use arithmetic operators with NULL, and you must check for it using IS NULL.

Q11. What do we mean by a database transaction?

A: A **database transaction** is a single unit of work that contains one or more SQL statements. A transaction must be treated as **atomic** (all or nothing); either all statements within it succeed and are permanently saved (**COMMIT**), or if any fail, the entire group is canceled (**ROLLBACK**).

Q12. How is an INNER JOIN different from a NATURAL JOIN?

A: An **INNER JOIN** requires you to explicitly specify the columns used to link the tables (e.g., ON table1.id = table2.fk_id). A **NATURAL JOIN** automatically joins tables based on all columns they share that have the same name, which can sometimes lead to unintended results if column names overlap.

Q13. How can you join a table with itself?

A: You join a table with itself using a **SELF JOIN**. This is done by specifying the table twice in the FROM clause and giving each instance a different **alias** (e.g., SELECT * FROM Employees e1 JOIN Employees e2...). This is useful for finding hierarchical data, like finding an employee and their manager from the same table.

Q14. What is a trigger, and how do you create one in MySQL?

A: A **trigger** is a stored SQL routine that automatically executes (or "fires") when a specific event occurs in a table (like an INSERT, UPDATE, or DELETE). You create one by defining the event, the action (BEFORE or AFTER), and the SQL statements to run.

Q15. What is a stored procedure, and how do you make one?

A: A **stored procedure** is a prepared piece of SQL code that you can save in the database and execute later. You make one by using the CREATE PROCEDURE statement, which allows you to define complex logic, receive input parameters, and return results.

Q16. What is a cursor, and how is it used in MySQL?

A: A **cursor** is a database control structure used to traverse and process the records returned by a query one row at a time. It is typically used inside stored procedures or functions when you need to perform complex logic on a row-by-row basis rather than on the entire set at once.

Q17. What is a user-defined function (UDF), and how do you write one?

A: A **user-defined function (UDF)** is a routine that performs an action and returns a single, scalar value. You write one using the CREATE FUNCTION statement, defining the function name, input parameters, and the data type of the return value.

Q18. What are aggregate functions in SQL?

A: **Aggregate functions** perform a calculation on a set of rows and return a single value summarizing the data. Common examples include **COUNT** (to count rows), **SUM** (to add values), **AVG** (to calculate the average), **MIN**, and **MAX**.

Q19. How is the WHERE clause different from the HAVING clause?

A: The **WHERE** clause is used to filter individual rows **before** any aggregation (GROUP BY) occurs. The **HAVING** clause is used to filter groups of rows **after** aggregation has occurred. You cannot use aggregate functions inside a WHERE clause.

Q20. What are indexes in SQL and why do we use them?

A: **Indexes** are special lookup tables that the database uses to speed up data retrieval, similar to an index in a book. We use them to quickly find data without scanning every row in a table. They are essential for columns used in WHERE clauses, JOIN conditions, and ORDER BY clauses.

Q21. How can you check which indexes a query is using?

A: You can check which indexes a query is using by preceding the SELECT statement with the **EXPLAIN** keyword (e.g., EXPLAIN SELECT * FROM table...). The output shows the database's

execution plan, including which indexes it considered and which one it ultimately chose to use.

Q22. Can you put an index on a view? If yes, how?

A: You cannot put an index on a view directly. However, in some systems (like SQL Server), you can create an **Indexed View** (or materialized view), which saves the view's results to disk and indexes the data, allowing the view to be queried faster.

Q23. You have two tables (shop_1 and shop_2) with the same columns. How do you find customers who appear in both tables?

A: You find customers who appear in both tables using the **INTERSECT** operator (supported by some databases like Oracle/SQL Server). If INTERSECT is not supported (as is the case in MySQL), you can achieve the same result using an **INNER JOIN** or the **IN** operator between the two tables.

Q24. What is the difference between INNER JOIN and OUTER JOIN?

A: An **INNER JOIN** returns only rows that have matching values in **both** tables. An **OUTER JOIN** (LEFT, RIGHT, or FULL) returns all rows from one or both tables, plus any matching rows from the other, filling in NULL where there is no match.

Q25. What is the difference between WHERE and HAVING?

A: **WHERE** filters individual records *before* aggregation (GROUP BY), and **HAVING** filters groups of records *after* aggregation. (This is a repeat of Q19, but given the user included it again, I will re-answer it).

Q26. What is the difference between UNION and UNION ALL?

A: **UNION** combines the result sets of two or more SELECT statements and automatically removes all duplicate rows. **UNION ALL** combines the result sets but includes all duplicate rows, making it generally faster since it skips the deduplication step.

Q27. What is the difference between DELETE and TRUNCATE?

A: **DELETE** is a DML command that removes rows one by one, logs the operation, and can be rolled back. **TRUNCATE** is a DDL command that removes all rows quickly by deallocating the entire table space, is much faster, and **cannot be rolled back**.

Q28. What is the difference between PRIMARY KEY and UNIQUE?

A: The **PRIMARY KEY** uniquely identifies every row and must be non-NULL. The **UNIQUE** constraint ensures all values are distinct but can allow one NULL value. (This is a repeat of Q8, but given the user included it again, I will re-answer it).

Q29. What is the difference between DROP and TRUNCATE?

A: **DROP** is a DDL command that removes the entire table structure (schema, indexes, and data) permanently from the database. **TRUNCATE** is a DDL command that removes all the data quickly but leaves the table structure (schema and indexes) intact.

Q30. What is the difference between CHAR and VARCHAR?

A: **CHAR** is a fixed-length string data type; if you define CHAR(10) and store 'cat' (3 characters), it still uses 10 bytes of storage. **VARCHAR** is a variable-length string; if you define VARCHAR(10) and store 'cat', it uses only 3 bytes plus 1 or 2 bytes overhead.

Q31. What is the difference between IN and EXISTS?

A: **IN** checks if a value matches any value in a list of values, often used with static lists or small subqueries. **EXISTS** checks if a subquery returns any row at all, and is generally more efficient for subqueries that return a very large result set.

Q32. What is the difference between using JOIN and using a SUBQUERY?

A: A **JOIN** is used to combine columns from two or more tables into a single result set, typically used for retrieving related data. A **SUBQUERY** (or inner query) is used to retrieve data that will be used as a condition for the main query, often used for filtering data based on complex criteria.